

Senior Design Autobot Project — Navilott: Part One

Mike Orduna

Ana Carlos

Ignacio Rubio

October 8, 2026

Contents

1	Abstract	1
2	Overview	1
2.1	Purpose of This Report	1
2.2	The Team	1
2.3	Development Process	1
2.4	How to Read This Report	1
3	Problem	1
3.1	Objective	1
3.2	Constraints	1
3.3	The Course	2
3.4	Requirements	2
I	Design	3
4	Architecture	3
4.1	Single-Board Design	3
4.2	Additional Hardware	3
4.3	Software Language	3
4.4	Tooling	3
4.5	Software Architecture	3
5	Capture	3
5.1	Camera Path	3
5.2	Design Decisions	4
5.3	Failure Handling	4
5.4	Open Questions	4
6	Perception	4
6.1	Approach	4
6.2	Stage Order	4
6.3	Lane Boundaries and Offset	4
6.4	Stop Lines	4
6.5	Traffic Lights	4
6.6	Stop Signs	5
6.7	Debugging Perception	5
7	Estimation	5
7.1	Role	5
7.2	Lane Filter	5
7.3	Heading Tracker	5
7.4	Voting	5

7.5	The Packet	5
8	Navigation	5
8.1	Scope and Attribution	5
8.2	The Contract	6
8.3	Driving Rules	6
8.4	Measured Robot Response	6
8.5	Intersections and Route	6
II	Evidence	7
9	Testing	7
9.1	Scope	7
9.2	Testing Without the Robot	7
9.3	The Suite	7
9.4	What Testing Proves	7
10	Diagnostics	7
10.1	Why Instrument the Pi	7
10.2	What Is Recorded	7
10.3	Observer Effect	8
10.4	What It Showed	8
11	Verification	8
11.1	What Counts as Verified	8
11.2	Routines	8
11.3	Checks by Requirement	8
11.4	Whole-Robot Checks	8
12	Results	8
12.1	Status by Requirement	8
12.2	Routine Results	9
12.3	Course Runs	9
12.4	Measured Robot Response	9
III	Assessment	10
13	Limitations	10
13.1	Requirements Not Met	10
13.2	Requirements Not Yet Shown	10
13.3	Departures from the PRD	10
13.4	Limits of the Design	10

14 Discussion	10
14.1 What Worked	10
14.2 What Didn't	10
14.3 What I Would Do Differently	10
14.4 What Remains	10
15 Conclusion	11
 IV Supplements	 12
A Hardware	12
B Parameters	12
B.1 Capture	12
B.2 Perception	12
B.3 Estimation	12
C The Estimation Packet	12
D Repository	13
D.1 Layout	13
D.2 Commands	13

1 Abstract

[TODO: Abstract.]

2 Overview

2.1 Purpose of This Report

[TODO: Why write a self-evaluation at all? Who is the reader (you, an interviewer, the team)? What period of work does it cover, and as of what date and commit?]

2.2 The Team

[TODO: Who owned which subsystem? What did each contribute to the parts this report covers?]

2.3 Development Process

[TODO: How was the work actually done: what did you design and write yourself, what came from AI-assisted sessions, and how did you review and verify that work?]

2.4 How to Read This Report

[TODO: A short map of the parts. Write this last.]

3 Problem

3.1 Objective

[TODO: What must the robot do, in one paragraph? Which PRD rules matter most to vision: right-lane driving, the exact traffic-light rules (green to enter, yellow only once inside, never on red), the stop sign at the exit?] [TODO: Obstacle avoidance: The PRD's executive summary states it as a must. Was it formally downgraded? By whom, and when?]

3.2 Constraints

[TODO: Which constraints actually shaped your design decisions, and which were just boxes to tick? Consider: \$100 BOM, the PCB requirement, 12 V limit, size (10 x 11 x 11 cm), local processing only, start button and 5 s countdown, elapsed time in tenths of a second.] [TODO: Battery display: the PRD asks the power system to alert navigation. Why was battery life moved to the display?]

3.3 The Course

[TODO: Describe the course the robot was built and tested on. Where does it differ from the PRD's course, and does each difference make the vision problem easier or harder?]

Table 1: [TODO: caption]

PRD	Team course
[TODO: rows]	

3.4 Requirements

[TODO: Turn the PRD's prose into requirements that can each be checked one way. For each: is it straight from the PRD, or derived (something vision needs so a PRD rule can be met)? What numbers will you hold yourself to, and why those numbers?] [TODO: The PRD's validation guidance asks for each object's detection field (range, angles, minimum size). Its own requirement, or part of the detection ones?]

Table 2: [TODO: caption]

ID	Requirement	Source
[TODO: rows]		

Part I

Design

4 Architecture

4.1 Single-Board Design

[TODO: How was the architecture chosen — what were some of the options considered and why were they not chosen over the final design?]

4.2 Additional Hardware

[TODO: What sort of peripherals were used and why? What were the tradeoffs considered? How does each peripheral fit into the overall architecture?]

4.3 Software Language

[TODO: What programming language was used for the software? Why was this language chosen over others? When it came to integration, what were the tradeoffs?]

4.4 Tooling

[TODO: What tools were used to develop the software? What was gained and lost by using these tools? How did the tools affect the architecture?]

4.5 Software Architecture

[TODO: After deciding on the main architecture and hardware peripherals to use, how was the software architecture designed? What were some of the key decisions made and why? How did the software evolve over time and what changes, if any, were made to the architecture?]

5 Capture

5.1 Camera Path

[TODO: How does light become a frame in Python? Which stages does the frame pass through, and which of them did you configure?]

5.2 Design Decisions

[TODO: Which capture decisions mattered, and what would have gone wrong without each? Consider the pinned sensor mode, the output size, keeping only the newest frame, flipping in GStreamer, and who assigns frame ids and timestamps.]

5.3 Failure Handling

[TODO: What happens when a frame is dropped, and when the camera dies? Which requirement does this serve, and how do you know it works?]

5.4 Open Questions

[TODO: What about capture is still unmeasured or uncertain? For example, the delay from exposure to timestamp, or exposure settings under course lighting.]

6 Perception

6.1 Approach

[TODO: Why classical computer vision rather than a learned model? What about this course made that a reasonable bet, and what did you give up?]

6.2 Stage Order

[TODO: What are the stages, in order, and why that order? Why is there no memory between frames at this phase?]

6.3 Lane Boundaries and Offset

[TODO: How are lane lines found, and how does a set of lines become one steering error? What does the sign of the offset mean? What happens when only one line is visible?]

6.4 Stop Lines

[TODO: How is a stop line told apart from a lane line? How is its distance measured, and in what units?]

6.5 Traffic Lights

[TODO: What made traffic lights hard on this course? How did the approach change once you found the lights were bare LEDs (glow mode)?]

6.6 Stop Signs

[TODO: How are stop signs detected, and how well tuned is the detector on real course frames?]

6.7 Debugging Perception

[TODO: How do you see why a frame failed? How do you see why a gate failed? Why would it be important to see this at all?]

7 Estimation

7.1 Role

[TODO: Where is the line between perception and estimation, and why draw it there?]

7.2 Lane Filter

[TODO: How is a raw offset smoothed, and when is a frame not trusted? What do vision, hold and stale mean, and why those thresholds?]

7.3 Heading Tracker

[TODO: What does the IMU add while vision is lost? What does heading actually measure, and what are its limits?]

7.4 Voting

[TODO: Why vote across frames for traffic lights, stop signs and stop lines? Why a window of three?]

7.5 The Packet

[TODO: What does navigation receive each frame? Which fields matter most? (Full field list in Appendix C.)]

8 Navigation

8.1 Scope and Attribution

[TODO: What is the role of navigation, and how was the design chosen? What was considered and what were the tradeoffs?]

8.2 The Contract

[TODO: What is the interface between vision and navigation? What must a navigator always do, and what must it never do? Why fix this before navigation existed?]

8.3 Driving Rules

[TODO: How does the robot respond to each light state, stop signs, stop lines and a lost lane? Do these rules match the PRD?]

8.4 Measured Robot Response

[TODO: How the robot actually responds to commands? How does that compare to the contract?]

8.5 Intersections and Route

[TODO: How is a turn sequenced, and how does the robot know which way to go at each intersection?]

Part II

Evidence

9 Testing

9.1 Scope

[TODO: What does testing check that verification doesn't, and the reverse? Where is the line between the two in this report?]

9.2 Testing Without the Robot

[TODO: How can the pipeline run with no robot attached? Which design choices made that possible, and when were they made?]

9.3 The Suite

[TODO: How large is the suite, what does it cover, and how long does it take to run? Which tests need the robot, and how are they selected?]

9.4 What Testing Proves

[TODO: Which requirements can software tests actually verify, and which can they not? Where did tests catch real bugs? Where did a passing suite hide a problem that only showed on the course?]

10 Diagnostics

10.1 Why Instrument the Pi

[TODO: What can go wrong on the Pi that the pipeline's own logs can't show? Why did that become worth building tools for?]

10.2 What Is Recorded

[TODO: Which recorders exist, and what question does each answer? Which ones actually changed a decision?]

Table 3: [TODO: caption]

Recorder	Question it answers
[TODO: rows]	

10.3 Observer Effect

[TODO: How do you know the recorder doesn't change the run it measures?]

10.4 What It Showed

[TODO: What did diagnostics reveal about frame rate, load, throttling or scheduling? Should the numbers go here, or only in Results?]

11 Verification

11.1 What Counts as Verified

[TODO: What has to be true before you call a requirement verified? Which hardware does it have to pass on, and do older results (IMX219) count?]

11.2 Routines

[TODO: What is a routine, and why run hardware checks this way instead of ad hoc? What does a request card fix in advance, and what does each run record?]

11.3 Checks by Requirement

[TODO: For each requirement: one check, with its pass criteria. Which requirements have no check defined yet?]

Table 4: [TODO: caption]

ID	Check and pass criteria
[TODO: rows]	

11.4 Whole-Robot Checks

[TODO: Which routines test the robot as a whole rather than a single requirement (figure-eight, course run, power profile ...)? What do they add?]

12 Results

12.1 Status by Requirement

[TODO: What grade does each requirement get, and on what evidence? Decide your grading scale first (e.g. Pass / Fail / Partial / Open) and define it in the caption.]

Table 5: [TODO: caption, with the grading scale]

ID	Grade	Evidence
[TODO: rows]		

12.2 Routine Results

[TODO: One row per routine run: routine, date, commit, trials, verdict, key number.]

12.3 Course Runs

[TODO: What happened when the robot ran the course? Early runs drifted right and failed at intersections. Has the stop-line fix resolved that?]

12.4 Measured Robot Response

[TODO: If the maneuver numbers live here rather than in Navigation, put them here.]

Part III

Assessment

13 Limitations

13.1 Requirements Not Met

[TODO: Which requirements fail, and what does each failure mean for how the robot drives?]

13.2 Requirements Not Yet Shown

[TODO: Which requirements have no hardware evidence either way? What is blocking each?]

13.3 Departures from the PRD

[TODO: Where does the robot's behavior differ from what the PRD asks?]

13.4 Limits of the Design

[TODO: What can this design never do well, however well it's tuned?]

14 Discussion

14.1 What Worked

[TODO: Which decisions would you make again, and what evidence says they were right?]

14.2 What Didn't

[TODO: Which decisions cost you, and when did you realize it?]

14.3 What I Would Do Differently

[TODO: With what you know now, what would you change in the design, the process, or how the team worked? How would you use AI-assisted development next time?]

14.4 What Remains

[TODO: What has to happen before Design Day, in priority order, and why that order?]

15 Conclusion

[TODO: If a reader only reads this section, what three things should they take away? What the pipeline is; how it measured against its requirements; what matters most next.]

References

G. Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000.

Mike Orduna, Ignacio Rubio, and Ana Carlos. Navilott Vision. https://github.com/Simbyote/Navilott_Vision, 2026.

Texas State University. AutoBots product requirements document. Technical report, Ingram School of Engineering, Texas State University, January 2026. Sponsored by Jeff Stevens and Mark Welker. Version 0.5.

Part IV

Supplements

A Hardware

[TODO: Parts list and pin map, for reference only. The reasons behind each part belong in Architecture.]

Table 6: [TODO: caption]

Part	Role	Interface
[TODO: rows]		

Table 7: [TODO: caption]

BCM	Header	Use
[TODO: rows]		

B Parameters

[TODO: Which tuning values does a reader need on record to reproduce your results? Group by phase.]

B.1 Capture

[TODO: Sensor mode, output size, frame rate.]

B.2 Perception

[TODO: ROIs, and the gates that matter.]

B.3 Estimation

[TODO: Filter, hold, vote and confidence settings.]

C The Estimation Packet

[TODO: Every field navigation receives per frame: name, type, meaning, units and sign convention.]

Table 8: [TODO: caption]

Field	Type	Meaning
[TODO: rows]		

D Repository

[TODO: Where the code is, how it is laid out, and the commands a reader needs to reproduce a result.]

D.1 Layout

[TODO: One line per directory under vision_stack/src/.]

D.2 Commands

[TODO: The few commands that matter: setup, every-boot session, run, test, routines.]